Werkzeuge · Warm-up-Karten

Warm-up-Karten Welt 4 – Der Hafen

Kurze Papier-Aufgaben für den Einstieg jeder Doppelstunde („10' Papier-Warm-up, ohne Gerät"). Die Karten W4-01 bis W4-14 begleiten den Hafen (DS 24–29): Befehlsfolgen auf Stapel, Schlange und dynamischer Reihung mit Zustandsskizzen, Mischaufgaben, Casten bei den Abitur-Klassen, Leer- und Index-Abfragen, das Muster „unverändert lassen" (Hilfsstapel, Hilfsschlange, Rotieren, Umkehren), Löschen und sortiertes Einfügen in der dynamischen Reihung, Klammerprüfung und Postfix-Rechner sowie die Eignung einer Struktur.

So funktioniert's

- Karte ziehen (oder zugeteilt bekommen), **ohne Gerät** auf einem eigenen Blatt lösen.
- Selbst- oder Partnerkontrolle mit dem Lösungsteil (gleiche Nummer). Fehler ins Fehlerlogbuch – Warm-up-Karten sind Training ohne Punkte.
- Digital im **Warm-up-Trainer** des Algo Quest Lab (Menü „Warm-up", Welt-Filter „Welt 4"): erst auf Papier lösen, dann aufdecken und ehrlich einschätzen (Leitner-System, ohne Punkte).
- Boss-Training vor Boss 4: Karten W4-01, W4-05, W4-08, W4-09 und W4-11. Zusätzlich im Lab: Online-Boss **„Die Postkutschenstation"**.

Achtung Abi:

Im Abitur schreibst du alles von Hand und nutzt die Schnittstellen Stapel, Schlange und dynamische Reihung exakt nach EH §4. Laufzeitfehler musst du durch **explizite Abfragen** ausschließen: vor top/pop/head/dequeue „**Leer?**", vor getItem/setItem/delete „**Index?**" (0 bis getLength() − 1, bei insertAt bis getLength()). Soll eine Struktur **unverändert** bleiben, schichtest du über eine Hilfsstruktur um und stellst sie vor dem return wieder her. Rückgaben vom Typ Object werden **gecastet**, z. B. `(Kiste) s.pop()` oder `(Integer) q.dequeue()`.

Kartenübersicht

Nr.	Kategorie	Titel	Min.	Einsatz
W4-01	Tracetabelle	Der Kistenkran	7	DS 24 + Boss
W4-02	Fehler im Java-Code	Casten am Kai	6	DS 24
W4-03	Tracetabelle	Die Ladeliste wächst	6	DS 27
W4-04	Funktionsweise erläutern	Das Zugbuch	7	DS 25 (Kür)
W4-05	Methodenkopf und Signatur	Die richtige Struktur für den Hafen	6	DS 24 / 25 + Boss
W4-06	Lückencode	Inventur ohne Chaos	6	DS 26
W4-07	Struktogramm lesen	Der Wechselzauber	7	DS 26
W4-08	Methode implementieren	Die Lotsenliste	8	DS 26 + Boss
W4-09	Fehler im Java-Code	Der Ladekran verrutscht	8	DS 27 + Boss
W4-10	Tracetabelle	Sortiert verladen	7	DS 27
W4-11	Code ↔ Struktogramm	Die leichteste Kiste zuerst	7	DS 27 + Boss
W4-12	Tracetabelle	Klammern im Logbuch	7	DS 28
W4-13	Tracetabelle	Der Postfix-Rechner des Zollamts	7	DS 28
W4-14	Funktionsweise erläutern	Die Flut im Hafenbecken	8	DS 29 (Kür)

Druckhinweis: Kartenseiten einseitig drucken, an den gestrichelten Linien schneiden (2 Karten im Format A5 je Seite), ggf. laminieren. Der Lösungsteil ist für die Kontrollstation bzw. das Lehrerexemplar gedacht.

WARM-UP W4-01 Tracetabelle

Welt 4 – Hafen · ⏱ 7 min

Der Kistenkran

Am Kai arbeitet ein Kran mit einem Stapel s und einer Schlange q, beide vom Inhaltstyp Ganzzahl und anfangs leer.

a) Führe die Zeilen 3 bis 11 aus. Notiere nach jeder Zeile den Inhalt von s (oberstes Element zuerst) und von q (vorderstes Element zuerst) sowie die Ausgabe. Leere Strukturen kennzeichnest du mit –.

b) Erläutere, warum in Zeile 9 der Cast (Integer) nötig ist und was die Zeile insgesamt bewirkt.

c) Zeile 6 wird durch s.pop(); ersetzt. Gib an, in welcher Zeile es dann zu einem Laufzeitfehler kommt, und wie man ihn durch eine Abfrage ausschließt.

```
1  Stack s = new Stack();
2  Queue q = new Queue();
3  s.push(4);
4  s.push(7);
5  q.enqueue(s.pop());
6  s.push(2);
7  q.enqueue(9);
8  q.enqueue(s.top());
9  s.push((Integer) q.dequeue() + 1);
10 q.enqueue(q.dequeue());
11 println(s.top() + " " + q.head());
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-01

WARM-UP W4-02 Fehler im Java-Code

Welt 4 – Hafen · ⏱ 6 min

Casten am Kai

Bibliothek „Niedersachsen“: Stack, Queue und DynArray sind nicht generisch – pop, top, dequeue, head und getItem liefern Object. Die Methode obenUndVorn soll das Gewicht der obersten Kiste von s und der vordersten Kiste von q addieren; eine leere Struktur trägt 0 bei. Beide Strukturen bleiben unverändert.

a) Gib für die Zeilen 2 bis 5 an, ob der Compiler sie akzeptiert. Begründe jeweils.

b) Gib eine korrigierte Fassung an, die auch bei leeren Strukturen ohne Laufzeitfehler arbeitet.

c) Jemand führt vorher s.push("Kiste") aus. Erläutere, warum der Compiler das nicht verhindert und was beim Aufruf deiner Methode geschieht.

```
1  public static int obenUndVorn(Stack s, Queue q) {
2      Kiste k = s.top();
3      int g1 = k.getGewicht();
4      int g2 = (Kiste) q.head().getGewicht();
5      return g1 + g2;
6  }
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-02

WARM-UP W4-03 Tracetabelle

Welt 4 – Hafen · ⏱ 6 min

Die Ladeliste wächst

Die Ladeliste d ist eine dynamische Reihung vom Inhaltstyp Zeichenkette. Die Nummerierung beginnt mit dem Index 0.

a) Gib nach jeder der Zeilen 2 bis 8 den Inhalt von d an (Index 0 zuerst) und notiere die Ausgabe von Zeile 9.

b) Die Anweisungen A, B und C werden jeweils einzeln direkt nach Zeile 9 ausgeführt. Entscheide begründet, ob es zu einem Laufzeitfehler kommt.

c) Formuliere die Abfragen („Index?“), mit denen vor d.getItem(i) bzw. vor d.insertAt(i, x) ein Laufzeitfehler ausgeschlossen wird.

```
1  DynArray d = new DynArray();
2  d.append("Anker");
3  d.append("Seil");
4  d.insertAt(0, "Fass");
5  d.insertAt(2, "Netz");
6  d.setItem(1, "Boje");
7  d.delete(3);
8  d.insertAt(d.getLength(), "Kran");
9  println(d.getLength() + " " + d.getItem(1));
   ------------------------------------------
A  d.delete(d.getLength());
B  d.insertAt(4, "Mast");
C  String t = (String) d.getItem(d.getLength() - 1);
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-03

WARM-UP W4-04 Funktionsweise erläutern

Welt 4 – Hafen · ⏱ 7 min

Das Zugbuch

Das Zugbuch des Spiels speichert Spielzüge, damit man sie rückgängig machen und wiederholen kann (Undo/Redo). Es verwendet zwei Stapel vom Inhaltstyp Zeichenkette.

a) Ein neues Zugbuch erhält nacheinander die Aufrufe zug("N"), zug("O"), zug("S"), rueckgaengig(), rueckgaengig(), wiederholen(), zug("W"), wiederholen(), rueckgaengig(). Gib zu jedem Aufruf die Rückgabe (falls vorhanden) und den Inhalt beider Stapel an (oberstes Element zuerst).

b) Erläutere, warum zug den Stapel zurueckgenommen leert. Was würde der vorletzte Aufruf liefern, wenn diese Zeile fehlte?

c) Begründe, warum hier Stapel und keine Schlangen verwendet werden.

```
public class Zugbuch {
    private Stack erledigt = new Stack();
    private Stack zurueckgenommen = new Stack();
    public void zug(String z) {
        erledigt.push(z);
        zurueckgenommen = new Stack();
    }
    public String rueckgaengig() {
        if (erledigt.isEmpty()) {
            return "";
        }
        String z = (String) erledigt.pop();
        zurueckgenommen.push(z);
        return z;
    }
    public String wiederholen() {
        if (zurueckgenommen.isEmpty()) {
            return "";
        }
        String z = (String) zurueckgenommen.pop();
        erledigt.push(z);
        return z;
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-04

WARM-UP W4-05 Methodenkopf und Signatur

Welt 4 – Hafen · ⏱ 6 min

Die richtige Struktur für den Hafen

a) Entscheide jeweils begründet, ob ein Stapel, eine Schlange oder eine dynamische Reihung am besten geeignet ist:

(1) Schiffe warten vor der Schleuse und werden in der Reihenfolge ihrer Ankunft geschleust.

(2) Kisten werden in einen schmalen Container mit nur einer Tür geladen und später wieder ausgeladen.

(3) Die Ladeliste eines Frachters soll nach Gewicht sortiert bleiben; Kisten kommen hinzu und werden auch aus der Mitte entnommen.

b) Gib zu den beiden Signaturen jeweils den Java-Methodenkopf an (Klassenoperationen, Bibliothek Niedersachsen).

c) Gib zum Java-Methodenkopf unten die Signatur in Abi-Notation an. Erläutere, welche Information der Java-Kopf – anders als die Signatur – nicht enthält, und warum das beim Casten wichtig ist.

```
naechstesSchiff(schleuse: Schlange vom Inhaltstyp Schiff): Schiff
ladeliste(container: Stapel vom Inhaltstyp Kiste, grenze: Ganzzahl): dynamische
Reihung vom Inhaltstyp Kiste

public static boolean passt(DynArray liste, Queue wartend, double faktor)
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-05

WARM-UP W4-06 Lückencode

Welt 4 – Hafen · ⏱ 6 min

Inventur ohne Chaos

Die Hafenmeisterin zählt die schweren Kisten eines Stapels. Danach muss der Stapel genau so aussehen wie vorher.

a) Ergänze die Lücken (1) bis (5).

b) Ein Lehrling nimmt statt des Hilfsstapels eine Hilfsschlange (Lücke (4): hilf.enqueue(k), Lücke (5): s.push(hilf.dequeue())). Gib für s = A(3), B(9), C(5) (oben zuerst, Gewicht in Klammern) und grenze = 4 die Rückgabe und den Inhalt von s nach dem Aufruf an. Was ist schiefgegangen?

c) Begründe, warum man anzahlSchwerer nicht mit top() allein lösen kann.

```
// liefert die Anzahl der Kisten in s mit gewicht > grenze; s bleibt unveraendert
public static int anzahlSchwerer(Stack s, int grenze) {
    Stack hilf = ___(1)___;
    int anzahl = 0;
    while (___(2)___) {
        Kiste k = ___(3)___;
        if (k.getGewicht() > grenze) {
            anzahl++;
        }
        ___(4)___;
    }
    while (!hilf.isEmpty()) {
        ___(5)___;
    }
    return anzahl;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-06

WARM-UP W4-07 Struktogramm lesen

Welt 4 – Hafen · ⏱ 7 min

Der Wechselzauber

Die Operation umordnen arbeitet mit einer Schlange q vom Inhaltstyp Zeichenkette und einem Stapel. Hinweis: Die Division von Ganzzahlen ist als Division ohne Rest zu verstehen.

a) q enthält A, B, C, D, E (vorderstes Element zuerst). Gib den Inhalt von s und q nach der ersten Schleife, nach der zweiten Schleife und am Ende an.

b) Gib das Ergebnis für q = A, B, C, D an und beschreibe allgemein, was umordnen mit einer Schlange aus n Elementen macht.

c) Begründe, dass umordnen auch für eine leere Schlange ohne Laufzeitfehler arbeitet.

```
umordnen(q: Schlange vom Inhaltstyp Zeichenkette): nichts
  s ← erzeuge Stapel
  n ← 0
  solange nicht q.isEmpty()
    s.push(q.dequeue())
    n ← n + 1
  solange nicht s.isEmpty()
    q.enqueue(s.pop())
  für i von 1 bis n / 2
    q.enqueue(q.dequeue())
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-07

WARM-UP W4-08 Methode implementieren

Welt 4 – Hafen · ⏱ 8 min

Die Lotsenliste

Lotsen begleiten Schiffe nur in aufsteigender Reihenfolge ihres Tiefgangs. Die Schlange q vom Inhaltstyp Ganzzahl enthält die Tiefgänge (in dm) der wartenden Schiffe.

a) Implementiere istAufsteigend(q: Schlange vom Inhaltstyp Ganzzahl): Wahrheitswert. Sie liefert wahr, wenn kein Wert kleiner als sein Vorgänger ist (leere Schlange und ein einzelner Wert: wahr). q muss nach dem Aufruf unverändert sein.

b) Ein Mitschüler ersetzt in deiner Lösung die Zuweisung „aufsteigend = false;“ durch „return false;“. Gib für q = 4, 2, 7 (vorn zuerst) die Rückgabe und den Inhalt von q nach dem Aufruf an und erläutere das Problem.

c) Nenne die Stellen deiner Lösung, an denen Laufzeitfehler ausgeschlossen werden.

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-08

WARM-UP W4-09 Fehler im Java-Code

Welt 4 – Hafen · ⏱ 8 min

Der Ladekran verrutscht

Die Methode entferneLeichte soll aus der dynamischen Reihung d vom Inhaltstyp Kiste alle Kisten mit einem Gewicht unter grenze entfernen. Sie enthält zwei Fehler.

a) d enthält A(3), B(2), C(8), D(1), E(1), F(9) (Index 0 zuerst, Gewicht in Klammern). Verfolge den Aufruf entferneLeichte(d, 4) in einer Tabelle i | Länge | getItem(i) | Aktion | d bis zum Ende – eine Zeile an jeder mit „// → Tabellenzeile“ markierten Stelle. Was geschieht am Schluss?

b) Nimm an, die Schleifenbedingung ist zu i < d.getLength() korrigiert. Gib d nach dem Aufruf an und erläutere den zweiten Fehler.

c) Korrigiere die Methode auf zwei Arten: (1) mit einer rückwärts laufenden Schleife, (2) mit einer while-Schleife, die i nur erhöht, wenn nicht gelöscht wurde.

```
// soll alle Kisten mit gewicht < grenze aus d entfernen
public static void entferneLeichte(DynArray d, int grenze) {
    for (int i = 0; i <= d.getLength(); i++) {
        Kiste k = (Kiste) d.getItem(i);
        if (k.getGewicht() < grenze) {
            d.delete(i);
        }   // → Tabellenzeile
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-09

WARM-UP W4-10 Tracetabelle

Welt 4 – Hafen · ⏱ 7 min

Sortiert verladen

Die Ladeliste d (dynamische Reihung vom Inhaltstyp Kiste) ist anfangs leer. Nacheinander werden die Kisten P(5), Q(2), R(5), S(9) und T(1) mit einfuegenSortiert(d, …) eingefügt (Gewicht in Klammern).

a) Gib für jeden Aufruf den Wert von j beim insertAt und den Inhalt von d danach an (Index 0 zuerst) – eine Zeile an der mit „// → Tabellenzeile“ markierten Stelle.

b) Begründe, warum R hinter P eingeordnet wird. Was ändert sich, wenn <= durch < ersetzt wird?

c) Erläutere, warum die beiden Teilbedingungen der while-Schleife nicht vertauscht werden dürfen. Gib einen Aufruf an, bei dem die vertauschte Fassung abstürzt.

```
public static void einfuegenSortiert(DynArray d, Kiste k) {
    int j = 0;
    while (j < d.getLength()
            && ((Kiste) d.getItem(j)).getGewicht() <= k.getGewicht()) {
        j++;
    }
    d.insertAt(j, k);   // → Tabellenzeile
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-10

WARM-UP W4-11 Code ↔ Struktogramm

Welt 4 – Hafen · ⏱ 7 min

Die leichteste Kiste zuerst

Die Operation entferneLeichteste entfernt die leichteste Kiste aus der Ladeliste und liefert sie zurück.

a) Implementiere die Operation nach dem Struktogramm in Java (Bibliothek Niedersachsen – denke an die Casts).

b) Gib für d = A(4), B(2), C(7), D(2) (Index 0 zuerst, Gewicht in Klammern) die Rückgabe und d nach dem Aufruf an. Welche Kiste wird bei Gleichstand entfernt? Woran liegt das?

c) Begründe, warum sich die Operation den Index der leichtesten Kiste merkt und nicht die Kiste selbst, und warum leichteste vor dem delete gespeichert wird.

```
entferneLeichteste(d: dynamische Reihung vom Inhaltstyp Kiste): Kiste
  falls d.isEmpty()
    dann: gib null zurück
    sonst: –
  minIndex ← 0
  für i von 1 bis d.getLength() – 1
    falls d.getItem(i).getGewicht() < d.getItem(minIndex).getGewicht()
      dann: minIndex ← i
      sonst: –
  leichteste ← d.getItem(minIndex)
  d.delete(minIndex)
  gib leichteste zurück
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-11

WARM-UP W4-12 Tracetabelle

Welt 4 – Hafen · ⏱ 7 min

Klammern im Logbuch

Das Logbuch prüft Formeln auf korrekt geschachtelte Klammern ( ), [ ] und { }. Andere Zeichen werden übersprungen.

a) Verfolge klammernOk("{a[b(c)]d}("): Gib den Inhalt von s (oberstes Zeichen zuerst) an jeder mit „// → Tabellenzeile“ markierten Stelle an, also nach jedem Klammerzeichen, und die Rückgabe.

b) Gib die Rückgaben für "([)]", ")(" und "(()" an und nenne jeweils die return-Anweisung, die das Ergebnis liefert (1, 2 oder 3).

c) Erläutere, warum s.push(c) mit einem Zeichen funktioniert und warum beim Entnehmen (Character) gecastet wird.

```
public static boolean klammernOk(String t) {
    Stack s = new Stack();
    for (int i = 0; i < t.length(); i++) {
        char c = t.charAt(i);
        if (c == '(' || c == '[' || c == '{') {
            s.push(c);   // → Tabellenzeile
        } else if (c == ')' || c == ']' || c == '}') {
            if (s.isEmpty()) {
                return false;                       // (1)
            }
            char auf = (Character) s.pop();   // → Tabellenzeile
            if (!(auf == '(' && c == ')' || auf == '[' && c == ']'
                    || auf == '{' && c == '}')) {
                return false;                       // (2)
            }
        }
    }
    return s.isEmpty();                             // (3)
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-12

WARM-UP W4-13 Tracetabelle

Welt 4 – Hafen · ⏱ 7 min

Der Postfix-Rechner des Zollamts

Das Zollamt rechnet in Postfix-Schreibweise (Operanden vor dem Rechenzeichen; einstellige Operanden, keine Leerzeichen, Ausdruck gültig).

a) Verfolge postfix("52-3*4+"): Gib den Inhalt von s (oberstes Element zuerst) an jeder mit „// → Tabellenzeile“ markierten Stelle an, also nach jedem Zeichen, und die Rückgabe.

b) Gib den Ausdruck in der üblichen Schreibweise mit Klammern an. Was liefert der Aufruf, wenn man die Zeilen mit rechts und links vertauscht (zuerst entnommener Wert = links)?

c) Gib einen ungültigen Ausdruck an, bei dem ein Laufzeitfehler auftritt, und nenne die Stelle. Wie müsste man die Methode absichern?

```
public static int postfix(String a) {
    Stack s = new Stack();
    for (int i = 0; i < a.length(); i++) {
        char c = a.charAt(i);
        if (c >= '0' && c <= '9') {
            s.push((int) c - (int) '0');   // → Tabellenzeile
        } else {
            int rechts = (Integer) s.pop();
            int links = (Integer) s.pop();
            int erg = links * rechts;                  // '*'
            if (c == '+') { erg = links + rechts; }
            if (c == '-') { erg = links - rechts; }
            s.push(erg);   // → Tabellenzeile
        }
    }
    return (Integer) s.pop();
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-13

WARM-UP W4-14 Funktionsweise erläutern

Welt 4 – Hafen · ⏱ 8 min

Die Flut im Hafenbecken

Die Flut läuft ins Hafenbecken: Von S aus werden alle erreichbaren Felder '.' markiert – ohne Rekursion, mit einem Speicher für Positionen (z|s): einer Schlange oder einem Stapel. Markiert wird beim Hineinlegen mit einer laufenden Nummer (S bekommt keine).

a) Führe fluten mit einer Schlange aus: Nummeriere die Felder in der Reihenfolge ihrer Markierung und gib den Inhalt der Schlange nach der dritten Entnahme an (vorn zuerst).

b) Führe fluten mit einem Stapel aus und nummeriere die Felder ebenso.

c) Erläutere, warum beide Varianten dieselben Felder markieren, aber in anderer Reihenfolge. Welche Variante breitet sich „wie eine Welle“ aus? Warum wird schon beim Hineinlegen markiert?

```
fluten(z: Ganzzahl, s: Ganzzahl): Ganzzahl
  nummer ← 0
  lege (z|s) in den Speicher
  solange der Speicher nicht leer ist
    (z|s) ← entnimm aus dem Speicher   // dequeue bzw. pop
    für jeden Nachbarn (nz|ns) von (z|s): oben, rechts, unten, links
      falls (nz|ns) im Feld und karte[nz][ns] = '.'
        dann: nummer ← nummer + 1
              karte[nz][ns] ← nummer   // markieren
              lege (nz|ns) in den Speicher
        sonst: –
  gib nummer zurück
```

```
     s:  0 1 2 3
z = 0:   . . . .
z = 1:   S . # .
z = 2:   . . . .
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W4-14